Open-Source Pulse

Lean big data scope and website implementation plan

Based on a review of github.com/shashwat230710/openSource_pulse. Prepared October 22, 2026.

> **The one rule.** A feature stays only if it (a) processes data at scale, (b) demonstrates a Spark, Parquet, GraphFrames or MLlib technique with *measured* evidence, or (c) displays a real pipeline output. Everything else is cut. The website shows only numbers exported from a real run: if it is not in the run manifest, it does not appear on the site.

1. Final scope at a glance

The current project has 7 website tabs, 9 benchmark experiments, 5 models, 2 serving back ends, an AI-insight endpoint and a simulator. The lean version keeps the parts that show big data engineering and removes the rest.

Layer	Keep (lean version)	Cut
Data	GH Archive, real run on T1 (Jan 1 to Mar 31, 2024; 2,184 hourly files). T0 (1 week) for tests.	The 18-month / 1.48B-row claims unless you actually run T2.
Storage	Bronze / silver / gold Parquet (zstd), explicit schema, partition by date.	Extra compaction and sorting work.
Spark tuning	6 measured experiments (Section 3).	E8 engine comparison, streaming replay, small-files study, salting, approxQuantile, standalone cluster.
Graph	Point-in-time co-star graph, PageRank + label propagation, degree-cap guard.	Connected components, triangle count, extra graph analytics.
ML	4 models: growth-ratio baseline, logistic regression, GBT, GBT + graph. Time split with embargo. PR-AUC and precision@k.	B0 baseline, rolling cross-validation, calibration curves, hand-built simulator.
Serving	Spark exports static JSON; the website reads it.	FastAPI, DuckDB, Express server, Docker files, Gemini endpoint, fake pipeline runner.
Tests	One real leakage test on a small fixture, one schema test.	Tests that assert on lists defined inside the test.

2. Pipeline fixes to do first

These are prerequisites. The experiments and the website are only meaningful once they are done.

#	Fix	Why it matters
1	Add **jobs/common.py**: `build_spark()` and a loader that reads config/pipeline.yaml. Every job uses it.	The YAML is currently never read; tier dates are hardcoded in the Makefile.
2	Add the GraphFrames `--packages` coordinate matching Spark 3.5 to the Makefile. **Delete the fallbacks** in 04_graph.py and 06_train_eval.py so a missing graph raises an error.	Right now a failed import silently produces fake PageRank and fake communities.
3	Rebuild the graph **point-in-time**: for each snapshot week, use only stars from the previous 60 days up to that week. Start with every 4th week to bound cost. Join to training rows on (repo_id, snapshot_week). Compare M2 and M3 on the same rows.	The current graph uses all-time stars and the latest week, so it leaks the future into training. This is why the M3 gain cannot be trusted.
4	Save the trained `PipelineModel` in 06; load it in 07 and score held-out weeks only.	07_score.py currently uses a hand-weighted formula, not the model.
5	Add a **run logger** that writes `run_manifest.json` and per-stage rows, size and runtime.	This is the single source for the funnel and the website.
6	Write one **real leakage test**: on a tiny fixture, change future events, recompute features, assert past rows are unchanged.	Replaces the tautological test.

3. The six experiments that show big data work

Each experiment runs on a real sample from your data, records both numbers in results.csv, and keeps a Spark UI screenshot or an `explain("formatted")` output in `docs/evidence/`. Report whatever comes out, including disappointing results, with an explanation.

ID	Experiment	How to measure	Concept shown
B1	Storage format	Write the same sample as gz JSON, Parquet-snappy and Parquet-zstd. Record size on disk and full-scan time.	Columnar storage and compression
B2	Schema and pruning	Inferred vs explicit schema ingest time. Full scan vs date filter + selected columns; save the explain plan showing PartitionFilters and PushedFilters.	Partition pruning, predicate pushdown
B3	Aggregation grain	Window features over raw events vs over repo-week. Record shuffle size and runtime.	Reduce data before the shuffle
B4	AQE and skew	AQE off vs on. Compute the share of events from the top 100 repos and show the task-time spread in the Spark UI.	Shuffle partitions, data skew
B5	Scalability	Runtime vs data size (1 week, 1 month, 3 months) and vs cores (1, 2, 4, 8). Compute parallel efficiency.	Scaling curves
B6	Graph guard	Co-star self-join with and without the degree cap on a one-week sample. Record pair counts and runtime.	Quadratic join explosion

Headline result for the ML part. One table: baseline, logistic regression, GBT, GBT + graph, with PR-AUC and precision@20/50/100 on the held-out test weeks, plus the real feature importances. If the graph adds little, say so; a measured negative result is a valid finding.

4. Website plan: five pages, all driven by exported JSON

Vercel cannot run Spark, so the correct design is: Spark runs offline, exports small JSON files, and the website renders them. This is a valid big data serving pattern and it removes the need for any back end.

Page	What it shows	Data file
Overview	The problem, run facts (date range, hours ingested, raw events, cores, Spark version, run date), and the headline model result next to its baseline.	manifest.json, models.json
Pipeline and Scale	Bronze / silver / gold diagram and the funnel table: rows, size and runtime per layer.	funnel.json
Optimizations	One baseline-vs-optimized chart per experiment B1 to B6, a one-line explanation, and a link to the evidence file.	benchmarks.json
Graph	Co-star network of the top ~150 repos, top PageRank repos, community list. Layout precomputed in Python so the browser only draws it.	graph_sample.json
Predictions	Model comparison table, precision-recall curve, feature importances, and a week selector listing the top-K repos with the real outcome (hit or miss).	models.json, predictions.json

Removed from the site: Signals tab, Simulator, Run Pipeline runner, separate Architecture tab (merged into Pipeline and Scale), and AI insight.

JSON data contract

File	Shape (keep each file under 1 MB)
manifest.json	run_id, generated_at, git_commit, spark_version, cores, driver_memory, tier, date_range, hours_ingested, raw_events, raw_size_gb
funnel.json	[{stage, rows, size_mb, runtime_s}] for raw, bronze, silver, repo-day, repo-week, candidates
benchmarks.json	[{id, title, baseline_label, baseline_value, optimized_label, optimized_value, unit, note, evidence_path}] plus scaling series [{x, y}] for B5
models.json	[{id, name, pr_auc, roc_auc, p_at_20, p_at_50, p_at_100, features}], pr_curve per model, importances per model
graph_sample.json	nodes [{id, name, community, pagerank, stars_cum, x, y}], edges [{source, target, weight}]
predictions.json	[{week_start, rank, repo_name, score, stars_cum, stars_w0, stars_next_4w, label}], top 50 per test week

Site rules. (1) No number is typed into the React code. (2) If a JSON file is missing, the page shows "not generated yet", never placeholder data. (3) Every page footer shows run_id and generated_at. (4) Export repository names only, never user logins.

5. Website implementation steps

Step	Work	Done when
W1 Strip	Delete server.ts, api/, the Gemini dependency, src/data/pulseData.ts and the Simulator and Pipeline tabs. Reduce vercel.json to a static Vite build with an SPA rewrite.	npm run build produces a static site with no server code.
W2 Foundation	Split the 1,989-line App.tsx into five page components. Add types.ts matching the data contract and a `useJson(path)` hook with loading and empty states.	Each page renders from a sample JSON file.
W3 Pages	Build in this order: Overview, Pipeline and Scale, Optimizations, Predictions, Graph (most effort, so last).	All five pages read real exported files.
W4 Charts	Use a light chart library or plain SVG for bars and curves. For the graph, export precomputed (x, y) positions from Python (networkx) and draw SVG.	No heavy runtime layout in the browser.
W5 Deploy	Commit the exported JSON into public/data, deploy on Vercel, and check that every number on the site matches manifest.json and results.csv.	Site numbers equal pipeline numbers.

6. Implementation timeline

Effort estimates assume one person working part-time; scale them to your deadline.

Phase	Work	Output	Effort
0 Cleanup	Delete files in Section 7, rewrite the README honestly.	Smaller repo, accurate README	0.5 day
1 Harden	Pipeline fixes 1 to 6 from Section 2.	common.py, run logger, real leakage test	2 days
2 Real run	Download T1, run bronze to gold, capture the funnel from the logger.	run_manifest.json, funnel.json	2 days
3 Experiments	Run B1 to B6 on samples; save evidence.	results.csv, docs/evidence/	3 days
4 Graph + ML	Point-in-time graph, four models, save and apply the model.	models.json inputs, predictions	3 days
5 Export + site	Write 08_export.py, then website steps W1 to W5.	Deployed five-page site	3 days
6 Report	Rewrite report and slides from measured numbers, with your name.	REPORT.md, slides	2 days

> **If time runs short, do these first:** phases 0, 1, 2, then experiments B1, B3 and B5, then the baseline-vs-GBT comparison, then the Overview, Pipeline and Predictions pages. That alone gives a real, reproducible big data project.

7. File-level plan

Action	Files
Delete	.aistudio/, public/assets/aistudio/, metadata.json, jobs/__pycache__/, duplicate presentation PDF, server.ts, api/index.ts, src/data/pulseData.ts, docker/, jobs/serving_api/, the Gemini dependency, the hardcoded benchmark list in run_experiments.py
Rewrite	benchmarks/run_experiments.py (real measurements), jobs/04_graph.py (point-in-time, no fallback), jobs/06_train_eval.py (4 models, save model, no placeholders), jobs/07_score.py (use the saved model), tests/test_leakage.py (fixture-based), Makefile (graphframes package, tier-driven dates), README and docs/REPORT.md
Add	jobs/common.py, jobs/08_export.py, benchmarks/results.csv (generated), run_manifest.json (generated), docs/evidence/, requirements.txt (pinned pyspark 3.5.x), public/data/*.json, src/pages/*.tsx, src/types.ts, src/useJson.ts
Keep	jobs/01 to 03 (small edits to use common.py), jobs/05_features_labels.py, config/pipeline.yaml (now actually read), scripts/make_urls.sh

8. What to show and how to describe it

- State exactly what was run: tier, date range, number of events, machine, cores. A real 3-month run beats an unreproducible 18-month claim.
- Use the scaling curve (B5) to argue how the pipeline would behave on more data; do not present extrapolation as a measured result.
- Put your own name on the report and mark its status as it is, not as "production-ready".
- Describe the limits plainly: stars measure attention, not quality; breakout labels are rare; results are from one time window.
- Keep the repo evidence in one place (docs/evidence/): Spark UI screenshots, explain plans, run_manifest.json.

Next step I can help with: writing common.py, the real benchmark harness, the point-in-time graph job, or 08_export.py with the website data hook.